SESI PRAKTIKUM · SUB-CPMK 2

Implementasi 4 Pilar OOP dalam Python

Studi Kasus: Class Mahasiswa

Pemrograman Berorientasi Objek

Dosen Pengampu: Wahyu Cahyo Utomo, S.Kom., M.Cs.

TUJUAN

Tujuan Praktikum

1. Mengimplementasikan class Mahasiswa (NIM, nama, IPK) dengan Python.
2. Menerapkan enkapsulasi menggunakan atribut privat dan property.
3. Menerapkan pewarisan untuk membentuk turunan class Mahasiswa.
4. Menerapkan polimorfisme melalui method overriding.
5. Menerapkan abstraksi menggunakan modul abc.

PERSIAPAN

Tools & Environment

1 Python 3.10+

Pastikan Python terpasang. Cek dengan: python --version

2 Code Editor / IDE

VS Code, PyCharm, atau Jupyter Notebook - sesuai preferensi/ketentuan kelas

3 Terminal / Command Line

Digunakan untuk menjalankan file .py: python nama_file.py

4 File Kerja

Buat file latihan_oop.py di folder kerja praktikum masing-masing

REVIEW

Syntax Dasar Class di Python

```
class Mahasiswa:
    def __init__(self, nim, nama, ipk):
        self.nim = nim
        self.nama = nama
        self.ipk = ipk

    def tampilkan_info(self):
        print(f"{self.nim} - {self.nama} (IPK: {self.ipk})")

# Membuat object
m1 = Mahasiswa("E41210001", "Rani", 3.7)
m1.tampilkan_info()
```

`__init__`
Constructor, dijalankan otomatis saat object dibuat.

`self`
Merujuk ke object itu sendiri.

`Method`
Fungsi yang didefinisikan di dalam class.

PRAKTIK 1

Enkapsulasi

```
class Mahasiswa:
    def __init__(self, nim, nama, ipk):
        self.nim = nim
        self.nama = nama
        self.__ipk = ipk

    @property
    def ipk(self):
        return self.__ipk

    @ipk.setter
    def ipk(self, nilai):
        if 0.0 <= nilai <= 4.0:
            self.__ipk = nilai
        else:
            raise ValueError("IPK harus 0.0 - 4.0")
```

Tugas Anda

1. Ketik ulang kode di samping.
2. Coba set ipk = 5.0 lalu amati error yang muncul.
3. Coba akses m1._Mahasiswa__ipk secara langsung - diskusikan mengapa ini bisa terjadi.

PRAKTIK 2

Pewarisan

```
class MahasiswaS1(Mahasiswa):
    def __init__(self, nim, nama, ipk, judul_skripsi):
        super().__init__(nim, nama, ipk)
        self.judul_skripsi = judul_skripsi

    def info_skripsi(self):
        return f"Skripsi: {self.judul_skripsi}"

m2 = MahasiswaS1("E41210002", "Budi", 3.6, "Sistem Pakar")
print(m2.info_skripsi())
print(m2.nama, m2.ipk)   # atribut warisan dari induk
```

Tugas Anda

1. Buat class MahasiswaS2 yang juga mewarisi Mahasiswa dengan atribut tambahan judul_tesis.
2. Pastikan super().__init__() dipanggil dengan benar.
3. Uji apakah m2 tetap bisa mengakses method dari class induk.

PRAKTIK 3

Polimorfisme

```
class MahasiswaS1(Mahasiswa):
    def predikat(self):
        return "Cumlaude" if self.ipk >= 3.5 else "Reguler"

class MahasiswaS2(Mahasiswa):
    def predikat(self):
        return "Cumlaude" if self.ipk >= 3.75 else "Reguler"

daftar = [m2, MahasiswaS2("E41210003","Sari",3.8,"Tesis AI")]
for mhs in daftar:
    print(mhs.nama, '->', mhs.predikat())
```

Tugas Anda

1. Jalankan kode dan amati hasil predikat() berbeda meski nama method sama.

2. Tambahkan satu subclass baru dengan aturan predikat berbeda.

3. Diskusikan: apa yang terjadi jika predikat() tidak di-override?

PRAKTIK 4

Abstraksi

```
from abc import ABC, abstractmethod

class Mahasiswa(ABC):
    def __init__(self, nim, nama, ipk):
        self.nim, self.nama, self.ipk = nim, nama, ipk

    @abstractmethod
    def predikat(self):
        pass

# Mahasiswa("E41", "Rani", 3.7)  -> TypeError!
# Wajib melalui subclass konkret seperti MahasiswaS1
```

Tugas Anda

1. Ubah class Mahasiswa menjadi abstract class seperti contoh.

2. Coba instansiasi Mahasiswa(...) langsung - catat pesan error.

3. Pastikan semua subclass (MahasiswaS1, MahasiswaS2) mengimplementasikan predikat().

LATIHAN TERBIMBING

Gabungkan Semua Pilar

1. Rancang abstract class Mahasiswa (abstraksi) dengan atribut NIM, nama, dan IPK privat (enkapsulasi).

2. Buat 2 subclass, misal MahasiswaS1 dan MahasiswaS2, yang mewarisi Mahasiswa (pewarisan).

3. Implementasikan method predikat() pada tiap subclass dengan aturan berbeda (polimorfisme).

4. Buat list berisi beberapa object dan tampilkan info + predikat masing-masing dalam satu loop.

DEBUGGING

Kesalahan Umum & Solusinya

`AttributeError: object has no attribute '__ipk'`

Lupa bahwa atribut __privat mengalami name mangling. Akses via method/property, bukan langsung.

`TypeError: Can't instantiate abstract class`

Ada abstractmethod yang belum di-override di subclass. Cek kelengkapan implementasi.

`TypeError: __init__() missing argument`

Lupa memanggil super().__init__() dengan parameter lengkap di subclass.

`Method induk terpanggil, bukan method subclass`

Nama method override tidak persis sama dengan method induk (typo/beda parameter).